

Introduction

In modern baseball analytics, understanding what impacts hard hits is key to evaluating a player's performance and potential. A player's hard-hit percentage, or the proportion of balls hit at 95 mph or higher, is one of the strongest indicators of offensive success. Utilizing predictive modeling, we aim to determine what metrics best predict a batter's ability to consistently make hard contact. Such models could help teams identify emerging power hitters and refine player development strategies based on measurable swing outcomes. Our research question was: *Which type of model best predicts the relationship between measurable batting and pitch characteristics and a hitter's hard-hit percentage?*

Methods

We used the *MLB Batting Exit Velocity* dataset from 2015 to 2022. The data was originally collected from the Baseball Savant's Statcast, which tracks information on every batted ball using high-speed cameras and radar. It contains 1,983 entries and 20 features, including performance metrics such as launch_angle, max_distance, hard_hit_percentage, and more. Our target variable was the hard_hit_percentage, which is the percentage of batted balls with an exit velocity over 95 mph. After splitting the data into training and test sets, we cleaned the data by removing unnecessary columns (id, rank, player) along with hard_hit_95mph+, which was duplicated information of the target variable. We dropped nine rows containing missing values and standardized all numerical features with StandardScaler in order to ensure fair model comparisons and improve model stability. All models were trained using 5-fold cross-validation with preprocessing applied inside each fold to prevent data leakage, and a fixed random seed (42) was used throughout to ensure reproducibility.

We trained four predictive models; Penalized Linear Regression (Ridge) using a scaled pipeline with StandardScaler and tuning multiple α values; Support Vector Machine (SVM) with linear, polynomial, and RBF kernels and tuned hyperparameters (C, gamma, epsilon); Random Forest ensemble optimized the number of estimators, leaf size, and tree depth; Neural Network implemented in PyTorch with two hidden layers, optimized learning rate, and hidden dimensions using cross-validated hyperparameter search. Models' performance was assessed using the Coefficient of determination (R^2), measures the proportion of variance in hard-hit percentage, and Mean Squared Error (MSE), which measures the average distance between predicted and true values.

Results

Among the four models, the Penalized Ridge Linear Regression had the strongest predictor of hard-hit percentage. It produced the highest cross-validated R^2 (0.942) and the lowest MSE (3.632), indicating that the relationship between the features and the target variable

is mainly linear. Similarly, the Neural Network performed well with an R^2 of 0.940 and an MSE of 3.775. It was able to capture meaningful relationships between predictors and the target variable, though its performance was slightly lower than Ridge Regression. The SVM model performed likewise ($R^2 = 0.938$, $MSE = 3.882$) with a linear kernel and gave us permutation importance results highlighting key predictors like barrel_batted_ball_events. Lastly, the Random Forest ensemble had the lowest performance of the four with an R^2 of 0.935 and an MSE of 4.157. Overall, all of the models performed similarly well, but ridge regression had the most accurate and best predictor. This is also seen in the final test-set evaluation with Ridge achieving an R^2 of 0.935 and MSE of 3.684, showing its consistent performance.

	Best R^2	Best MSE	Best Hyperparameters
Penalized Linear Regression (Ridge)	0.942	3.632	$\alpha = 1$
Support Vector Machine	0.938	3.882	$C = 10$, $\epsilon = 0.5$, kernel = linear, $\gamma = \text{scale}$
Random Forest Ensemble	0.935	4.157	300 trees, max_depth=None, min_samples_leaf=1
Neural Network	0.940	3.775	hidden_dim = 64, lr = 0.01, 50 epochs
Ridge (Test-set)	0.935	3.684	$\alpha = 1$

Discussion

Overall, the results demonstrate that the simpler models, like Ridge Regression, outperformed the more complex methods. Ridge performed the best, probably because the relationships in the dataset appear to be more linear, allowing the model to capture them effectively without overfitting, and it works well in small, low dimensional datasets. Both Neural Network and SVM models performed fairly well, but the complexity of their approach may have impacted their performance; the complexity may have added unnecessary noise. The Random Forest model performed the weakest, indicating that tree based methods may struggle with the smooth, continuous relationships presented in these features. A key limitation is the size of the dataset, which may have limited the ability of the models to generalize effectively. Additionally, we learned that several input features such as barrel rate and max exit velocity are highly correlated with the target, potentially leading to a target leakage. This feature redundancy could be further studied to learn how to make the predictive model more robust.